mshqc: A C++20 Quantum Chemistry Engine for Orbital-Optimized Correlation Methods

10 August 2026

Summary

`mshqc` is a quantum chemistry library that implements a focused set of electronic structure methods in a compact, C++20-native core with a zero-overhead Python interface built on `nanobind` [@jakob2022nanobind]. The library covers restricted, unrestricted, and restricted open-shell Hartree–Fock references (`RHF`, `UHF`, `ROHF`) [@roothaan1951new; @pople1954self; @roothaan1960self] accelerated by DIIS convergence extrapolation [@pulay1980convergence; @pulay1982improved]; second- and third-order Møller–Plesset perturbation theory in both standard and orbital-optimized forms (`RMP2/UMP2/OMP2`, `RMP3/UMP3/OMP3`) [@moller1934note; @head1988mp2; @neese2009assessment; @pople1976theoretical; @bozkaya2011quadratically]; and analytical energy gradients with geometry optimization [@pulay1969abinitio; @pople1979derivative].

Electron repulsion integrals are evaluated via `libcint` [@sun2015libcint], with an optional Cholesky-decomposed representation [@beebe1977simplifications; @koch2003reduced] and Schwarz-based prescreening [@haser1989improvements] to control the memory footprint for larger basis sets. Tensor contractions in the correlated methods are dispatched to TBLIS [@matthews2017high], a transpose-free contraction library, and orbital rotations in the orbital-optimized MP2/MP3 variants are handled with a trust-region second-order SCF (SOSCF) scheme using an approximate Hessian.

Statement of Need

Mature quantum chemistry ecosystems such as PySCF and Psi4 provide broad method coverage, but their size and layered abstractions make them a heavy starting point for a developer who wants to prototype a specific class of method — in this case, orbital-optimized perturbation theory — without first navigating a large general-purpose codebase.

`mshqc` addresses this gap with a small, single-purpose C++20 core built directly

around TBLIS tensor contractions and Cholesky/density-fitted integrals, so a developer working on an orbital-optimization algorithm can reach the relevant kernel in one or two hops rather than several abstraction layers. Its current focus is a from-scratch implementation of orbital-optimized MP2 and MP3, using trust-region SOSCF for orbital rotation alongside the standard Hartree–Fock references and DIIS acceleration needed to support it. The package targets method developers and students who want a compact, inspectable implementation of these specific methods, rather than the production-scale breadth of a general-purpose package.

Software Design

`mshqc`'s core is written in C++20 with RAII-based memory management and is exposed to Python through `nanobind` [@jakob2022nanobind] bindings that add negligible call overhead. Molecular geometry and basis set handling follow standard conventions [@helgaker2000molecular], with basis function contractions sourced from the Dunning correlation-consistent basis set families [@dunning1989gaussian]. Point group detection and Petite List construction for symmetry-adapted integral handling follow standard group-theoretic references [@atkins1970tables].

Two-electron integrals are computed through `libcint` [@sun2015libcint]. For larger basis sets, `mshqc` offers a Cholesky-decomposed ERI representation [@beebe1977simplifications; @koch2003reduced], reducing storage from $O(N^4)$ to $O(N^3 M)$ with $M \ll N^2$, together with Schwarz-bound integral prescreening [@haser1989improvements].

Tensor contractions in the correlated methods are dispatched to TBLIS [@matthews2017high], which performs transpose-free contraction directly on arbitrarily strided tensors, avoiding the explicit transposition overhead that reshapes-based contraction typically incurs. Shared-memory parallelism across the SCF and correlated-method kernels is provided by OpenMP [@dagum1998openmp], via the `llvm-openmp` runtime. The build applies link-time optimization and, by default, tunes for the host CPU's instruction set (`-march=native`); this can be disabled at build time for portable binaries intended to run on hardware other than the build machine.

Implemented Methods

`mshqc` exposes a hermetic Python API covering the following layers of electronic structure theory, each grounded in the literature and validated against established reference values.

Molecular Infrastructure. Geometry and basis set management (`Molecule`, `BasisSet`) follows the conventions of Helgaker et al. [@helgaker2000molecular], with basis function contractions sourced from the Dunning cc-pVDZ family

[@dunning1989gaussian]. Point group detection and the Petite List construction for symmetry-adapted integral handling (`PointGroup`, `PetiteList`) are implemented following standard group-theoretic references [@atkins1970tables].

Integral Engines. Analytical evaluation of one- and two-electron integrals is delegated to `libcint` [@sun2015libcint]. Cholesky decomposition of the four-center ERI tensor [@beebe1977simplifications; @koch2003reduced] is provided via `CholeskyERI`, enabling $O(N^3M)$-scaling storage reduction where $M \ll N^2$. Schwarz-bound prescreening [@haser1989improvements] is enforced through the `Screening` class.

Self-Consistent Field (SCF). The library implements Restricted (`RHF`) [@roothaan1951new], Unrestricted (`UHF`) [@pople1954self], and Restricted Open-shell (`ROHF`) [@roothaan1960self] Hartree-Fock methods, all accelerated by the DIIS extrapolation scheme [@pulay1980convergence; @pulay1982improved].

Møller-Plesset Perturbation Theory. Second-order perturbation theory is provided in the `RMP2` and `UMP2` variants [@moller1934note; @head1988mp2] and the orbital-optimized `OMP2` [@neese2009assessment]. Third-order corrections are available via `RMP3` and `UMP3` [@pople1976theoretical] and the orbital-optimized `OMP3` [@bozkaya2011quadratically].

Analytical Gradients & Geometry Optimization. First-order energy derivatives (`GradientResult`) are evaluated analytically following the Pulay force formalism [@pulay1969abinitio] and the coupled-perturbed framework of Pople et al. [@pople1979derivative]. Geometry optimization (`OptResult`) is driven by these gradient evaluations.

Dependency Stack

The build system requires CMake >= 3.18 and a C++20-compliant compiler. The full dependency graph is managed hermetically via `conda-forge`:

- **Linear algebra & tensors:** `eigen` [@guennebaud2010eigen], `tblis` [@matthews2017high], `mkl`, `mkl-include`
- **Integrals & I/O:** `libcint` [@sun2015libcint], `hdf5` [@hdfgroup1997hdf5]
- **Memory allocator:** `jemalloc` (arena-based allocation to reduce fragmentation overhead)
- **Parallelism:** `llvm-openmp` [@dagum1998openmp]
- **Python interface:** `nanobind` [@jakob2022nanobind], `numpy` [@harris2020array], `scipy`, `pytest`

The CMake configuration applies link-time optimization (LTO/IPO) and, by default, hardware-specific instruction tuning (`-march=native`) during the `pip install -v -e .` invocation; this can be disabled for portable builds.

Acknowledgements

The author acknowledges `libcint` [@sun2015libcint] for analytical integral evaluation and TBLIS [@matthews2017high] for tensor contraction, both of which have been instrumental in building and validating `mshqc`'s numerical precision against reference values from PySCF and Psi4.

References